Measuring the Carbon Footprint of Programming Languages

Anuj Khadka
Computer Science and Business Analytics
Caldwell University
akhadka@caldwell.edu

Vladislav D. Veksler
Computer Science and Business Analytics
Caldwell University
akhadka@caldwell.edu

Darryl Aucoin
Computer Science and Business Analytics
Caldwell University
akhadka@caldwell.edu

Abstract

This project investigates how programming language choice affects the carbon footprint produced during a program execution. Although software is often seen as intangible, every program consumes energy, ultimately contributing to carbon emissions. As the digital system continues to grow, understanding the environmental impact of software has become increasingly important. This experiment studies how identical algorithms behave when implemented in different programming languages running on the same hardware. By measuring and estimating the carbon emissions, the project tends to explore whether programming language designs play an important role in software sustainability, or if it is mostly driven by heavy algorithms. This research combines the aspects of computer science and environmental science to better understand the hidden environmental cost of everyday computational tasks and encourage more sustainable and responsible software development practices.
Keywords: carbon footprint, software sustainability, energy consumption

1 Introduction

As the tech continues to evolve, software becomes an increaingly central part of our lives. From the apps on our phones to the software running our cars, it's everywhere. Everytime a piece of code runs, it triggers energy usage inside the CPU, GPU, and other hardware components [**greenuk_footprint**]. This subsequent usage of software adds up to the carbon emission a device generates [**thoughtworks_carbon_intensity**]. The carbon footprint of software is a growing concern, as it contributes to climate change and has a significant impact on our planet.

The scale of this concern is larger than most people realize. The information and communication technology (ICT) sector accounts for an estimated of 2.1% to 3.9% of the global greenhouse gas emissions [**ict_stat**]. The global data center energy consumption alone is projected to reach around 945TWh by 2030, which is a significant increase from 153TWh in 2005, 205TWh in 2018. [**iea_energy_consumption**, **masanet_energy_consumption**]. Unlike the visible emissions from vehicles, software carbon is invisible. There is no smoke when a program runs, yet every computation contributes indirectly to the same atmosphere. This invisibility is precisely what makes it dangerous. It does not trigger the same urgency that the physical pollution does, even as the number grows.

One of the most consequential and least-studied of those decisions is the choice of programming language. The carbon cost of a running program is not determined by a hardwar alone. It is also shaped by how efficient the language runtime executes instructions, manages memory, and utilizes the CPU.In a landmark empirical study, Pereira et al. [**10.1145/3136014.3136031**] benchmarked 27 programming languages across 10 problems drawn from the Computer Language Benchmarks Game [**clbg**], measuring energy consumption via Intel's Running Average Power Limit (RAPL) interface. They found that the choice of programming language can have a significant impact on energy consumption, with some languages consuming up to 2.5 times more energy than others for the same task. For example, they found that C and Rust were among the most energy-efficient languages, while Python and JavaScript were among the least efficient. Python consumed approximately 75.88 times more energy per benchmark than C while JavaScript consumed approximately 4.45 times more energy per benchmark than C [**10.1145/3136014.3136031**]. Since carbon emissions are directly proportional to energy consumption, these differences are not just technical concern, but also a carbon concern. A Python program consuming about 75 times more energy than an equivalent C program does not just run slower, it emits 75 times more carbon for every execution. Now, multiply that by the millions of times a piece of software runs, and the carbon cost becomes staggering.

Prior work in this area, including Pereira et al., reports energy consumption in joules or kilowatt-hours but none

of it completes the translation into carbon footprint. This is a critical gap. Energy consumption is the underlying driver, but carbon footprint is the ultimate impact. This unit is most directly relevent to the clomate policy and environmental decision making. Henderson et al. showed that the emissions can be reduced by up to 30 times just by running the experiments in the locations powered by more renewable energy sources [**10.5555/3455716.3455964**].

This paper presents a controlled empirical study comparing the energy consumption and the carbon footprint of six programming languages (C, Rust, Go, Java, Javascript, and Python) across six different algorithms and three imput sizes. The energy consumption is measured using Intel's RAPL (accessed through LibreHardwareMonitor [**librehardwaremonitor**]), and real-time carbon intensity data was fetched from Electricity Maps API [**elecritymap**] for the New York grid zone. stat analysis of pairwise langugage difference

2 Background

In the following sections, we describe the energy consumption model, carbon intensity assumptions, programming languages, and the implementations of the six algorithms used in this study.

2.1 Energy Consumption in Software

To quantify the energy consumed by a software program, we adopt the standard defination of energy as the capacity to do work.

$$E_{\text{energy}} = P_{\text{power}} \times t_{\text{time}} \tag{1}$$

where E is energy measured in Joules(J), Power is rate of energy consumption measured in Watts(W), and Time is the duration of execution measured in seconds(s)[**8323579**]. In the context of software execution, the T refers to the wall time, the elapsed time from the first instruction of the measured code path to the last, including CPU computation, memory access, and all other processor activity during execution [**carter2024**].

The energy consumed by a CPU package during the program execution can therefore be decomposed as the integral of instantaneos power draw over the execution window. Since power draw is not constant across languages or algorithms, a key finding of Pereira et al. [10.1145/3136014.3136031], energy efficiency cannot be inferred from execution time alone. A faster program is not necessarily a more energy efficient one. This encourages us to direct hardware level energy measurement via RAPL.

All experiments were conducted on a desktop with the following specifications: Windows 11 Enterprise 25H2 (Build 26200.8246) operating system, with 32.0 GB RAM, and an Intel Core `i7-14700K` CPU @ 3.40 GHz.

2.2 Carbon Intensity

To translate the energy measurements into carbon emissions, we apply the SCI framework.

2.3 Programming Language

The programming languages chosen for this study are C, Python, Javascript, Java, Go, and Rust. These languages were selected to represent a range of programming paradigms, performance characteristics, and popularity in the software development community. Each language has its own unique features and trade-offs that may influence the energy consumption and carbon footprint of the algorithms implemented in them [**decisionmodels**]. The performance characteristics of these languages are summarized in Table **??**.

Table 1: Programming languages, their performance characteristics, and popularity.

Language	Paradigm	Runtime	Version
C	Manual Memory, Compiled	GCC	14.3.0
Rust	Ownership, Compiled	Rustc	1.94.0
Python	Interpreted	CPython	3.12.10
Javascript	Dynamic, V8	Node.js	24.14.0
Java	OOP, JVM	OpenJDK	21.0.10
Go	GC	Go	1.26.1

C provides direct hardware access with no runtime overhead and serves as the energy baseline for all normalized comparisons.

Rust

Go

Java

JavaScript

Python

2.4 Algorithms

The algorithms were chosen to span a range of time complexities and computational patterns, including CPU-bound, memory-bound, and branching-heavy workloads. The six algorithms used in this study are summarized in Table **??**.

Table 2: Algorithms, their time complexities and computational patterns.

Algorithm	Time Complexity	Pattern
Summation	$O(n)$	Linear
Binary Search	$O(\log n)$	Logarithmic
Merge Sort	$O(n \log n)$	Divide & Conquer
Breadth-First Search (BFS)	$O(V + E)$	Graph Traversal
Hash Table	$O(n) - avg$	xxxxxx
Matrix Multiplication	$O(n^3)$	Dense Arithemetic

The pseudocode for each algorithm, as implemented indentically across all languages, is provided below. The implementations are designed to be as similar as possible across languages to ensure a fair comparison of their energy

consumption and carbon footprint.

Summation - This algorithm performs a single sequential sum over an array of integers. It is a simple CPU-bound with no branching and minimal memory access, making it a baseliune workload that reflects the idle runtime cost of the language.

Binary Search - This algorithm halves the search interval over a sorted array. Its time complexity is logarithmic, and it involves branching based on comparisons, which means very few operations are performed at any input size. So, the observed energy difference between languages may be more influenced by the runtime efficiency and overhead than the algorithmic complexity.

Breadth-First Search - This algorithm traverses a graph level by level from a source node. It is memory-bound. The primary cost is queue operations and random memory access patterns that can lead to cache misses. The energy consumption may be more sensitive to how efficiently the language runtime manages memory and data structures.

Merge Sort - This algorithm recursively divides the array into halves and merges them back in sorted order. Now, just to make sure that we don't get any RecursionError in Python, we will increase the recursion limit for merge sort.

Hash Table - This algorithm use open addressing with linear probing. Each operation is O(1) on average, but can be O(n) in the worst case if all keys collide. The energy consumption may be influenced by how well the language runtime optimizes for hash table operations and memory management. the "& 0xFFFFFFFFFFFFFFFF" is used to ensure that the hash result is always a 64bit integer, which is important for consistency across different platforms and languages. This way, we can compare the hash results from C and Python without worrying about differences in integer sizes or overflow behavior.

Matrix Multiplication - This algorithm uses the naive triple loop method to multiply two square matrices. It is a dense arithmetic workload with cubic time complexity, which means the number of operations grows rapidly with input size. At n=500 (our large input), it performs 125 million multiplications and additions, which is a significant computational load that can highlight differences in how languages handle CPU-bound tasks and memory access patterns.

3 Methodology

This section describes the experimental setup, the measurement protocol used to isolate algorithm energy consumption, the implementation constraints across all six languages, the input sizes used for each algorithm, and idk.

3.1 Experimental Setup

All the experiments were conducted on a single dedicated machine to eliminate any inter-machine power variability. The Intel's RAPL interface exposes hardware energy counters directly to software, enabling fine-grained CPU package energy measurement without externam instruments [**10.1145/3177754**]. Baseline idle power draw -

3.2 somehting...

The main challenge in cross-language energy measurement is to ensure that we are measuring the energy consumed by the algorithm itself, and not the overhead of the language runtime or other background process. To address this, we designed a custom measurement harness that follows a strict protocol to isolate the algorithm execution. The harness uses a stdin/stout handshake design in which the program initializes once and then loops, executing the algorithm on each trigger without restarting the process.

The measurement sequence is shown in the Listing **??**.

```
setup - not timed
print "ready" - not timed
harness newline - start timer
run command - measurement
print checksum - stop timer
flush output - not timed
```

Listing 1: Measurement sequence

The flush() call on line 7 is critical. Without it, the newline sits in a kernel pipe buffer and the program never unblocks, causing the measurement to hang indefinitely. On the program side, each language blocks on its respective stdin/stdout primitive until the trigger arrives, executes the algorithm, and writes the checksum. Listing **??** shows the blocking call used in each language.

```
// C
while (fgets(buf, sizeof(buf), stdin))  {
    run()}

// Python
for line in sys.stdin: run()

// JavaScript
rl.on('line', () => run())

// Java
while(br.readLine() != null) {run()}

// Go
for scanner.Scan() {run()}

// Rust
for line in stdin.lock().lines() {run()}
```

Listing 2: Stdin primitives in each language implementation

C uses fgets rather than scanf because it reads exactly one line including the new fgets and sys.stdin and readline

all blocks waiting for input until there is something to read from stdin/fgets. then reeads one line and returns it. So the harness will print "ready" and then wait for the next line. The next line is the signal to start the timer and run the command. After the command is done, the harness will print the checksum and stop the timer. This way we can measure the time taken by the command to run and also verify the output using checksum. For now, the command is "enter" btn, if I were to run the harness manually. But the verify file does this in the file itself.

Observation

```
- I can verify the thing now.
- The output that I get from the
baseline of C language can be
used to verify the time complexity
of the algorithms.
-
```

Table 3: C — RUN & VERIFY

Algorithm	Size	Output	Time	Status
summation	small	5050	0.0437ms	PASS
summation	mid	50005000	0.0406ms	PASS
summation	large	500000500000	0.4715ms	PASS
`binary_search`	small	99	0.0448ms	PASS
`binary_search`	mid	9999	0.0392ms	PASS
`binary_search`	large	999999	0.0835ms	PASS
`merge_sort`	small	100	0.0428ms	PASS
`merge_sort`	mid	10000	0.2459ms	PASS
`merge_sort`	large	1000000	24.7405ms	PASS
bfs	small	100	0.0430ms	PASS
bfs	mid	1000	0.3039ms	PASS
bfs	large	5000	7.2774ms	PASS
`hash_table`	small	64850	0.0433ms	PASS
`hash_table`	mid	649985000	0.2467ms	PASS
`hash_table`	large	64999850000	2.1579ms	PASS
`matrix_multiplication`	small	9175	0.0744ms	PASS
`matrix_multiplication`	mid	646700	2.5604ms	PASS
`matrix_multiplication`	large	10291750	47.1134ms	PASS

baseline power draw - 40w 5s sample
50reps each sample

3.2.1 Implementation-specific design decisions

Each language's implementation follows the stdin/stdout handshake protocol. The program prints `"ready"` on startup, then blocks waiting for a newline trigger from the measurement harness before executing the algorithm. The specific I/O primitives used, and the rationale for each choice, are described below:

C - fgets

Python - stdin/stdout

JavaScript - readline

Java - public static void main
buffered reader/writer

Go - bufio

Rust - `wrapping_mul` 2654435761

4 Results

5 Discussion

6 Conclusion

Acknowledgements

This paper uses AI for few code writing processes, such as generating initial code snippets and providing suggestions for code improvements.

add some more later in this